Desenvolvimento Web

Profª Sabrina B. Moreira

Recapitulando as últimas aulas…

Aula 3.1

Tipos de sistemas: centralizado x distribuído

- O sistema de folha de pagamento é **centralizado**, pois executa e processado tudo em **um único computador**.
- Entretanto, o site de vendas é considerado um sistema **distribuído**, pois parte do seu processamento ocorre em um computador e parte em outro, denominado servidor.
- Esse tipo de aplicação é organizado como um sistema **cliente-servidor**.

Internet: surgimento do hipertexto

- A internet é conhecida como a grande rede, e **interliga computadores de todo o mundo**.
- A partir das ideias de Tim Berners-Lee, surgiu o conceito de **hipertexto** que trouxe uma nova forma de organizar a informação.
- Permitia percorrer documentos por palavras destacadas no texto, chamado de **hyperlinks** ou **links**.

Internet: alguns marcos

1991	Surgimento da lista de discussão eletrônica WWW-talk, cujo foco é compartilhar experiências relacionadas com a linguagem HTML.
1992	Desenvolvimento do navegador Mosaic.
1993	Criação do navegador de texto Lynx.
1994	Estabelecimento da especificação HTML 2 (unificação das diferentes formas de HTML). Criação da empresa Netscape Communications Corporation. Surgimento do W3C (*World Wide Web Consortium*).
1995	Definição de novas marcações para o HTML. Criação do conceito de frames. Lançamento do navegador Internet Explorer 2.0.
1996	Lançamento do HTML versão 4.
1997	O HTML passa a incorporar tabelas, *applets*, subscritos, sobrescritos e texto combinado com imagens.
1998	Reformulação do HTML em XML.
1999	Lançamento do XHTML 1.0.
2004	Criação do grupo WHATWG (*Web Hypertext Application Technology Working Group*).
2007	Início do desenvolvimento do HTML 5, que incorpora TAGs para animações.

Fonte: MILETTO e BERTAGNOLLI, 2014

Internet: modelo cliente-servidor

- E como já vimos, a internet usa o modelo **Cliente-Servidor**.

- Onde o cliente solicita informações por um “endereço virtual” e o servidor, envia a resposta.

Conceitos de redes para desenvolvimento web

Aula 3.2

Domínio

- O **endereço virtual** é informado a um software chamado navegador (browser), e a partir disso, ele acessa a página solicitada.

- Domínio é o **nome que identifica um site na internet**.

Fonte: registromarcas

Domínio

Exemplo de registradores no Brasil:

- Registro.br
- GoDaddy

Quando você registra um domínio:

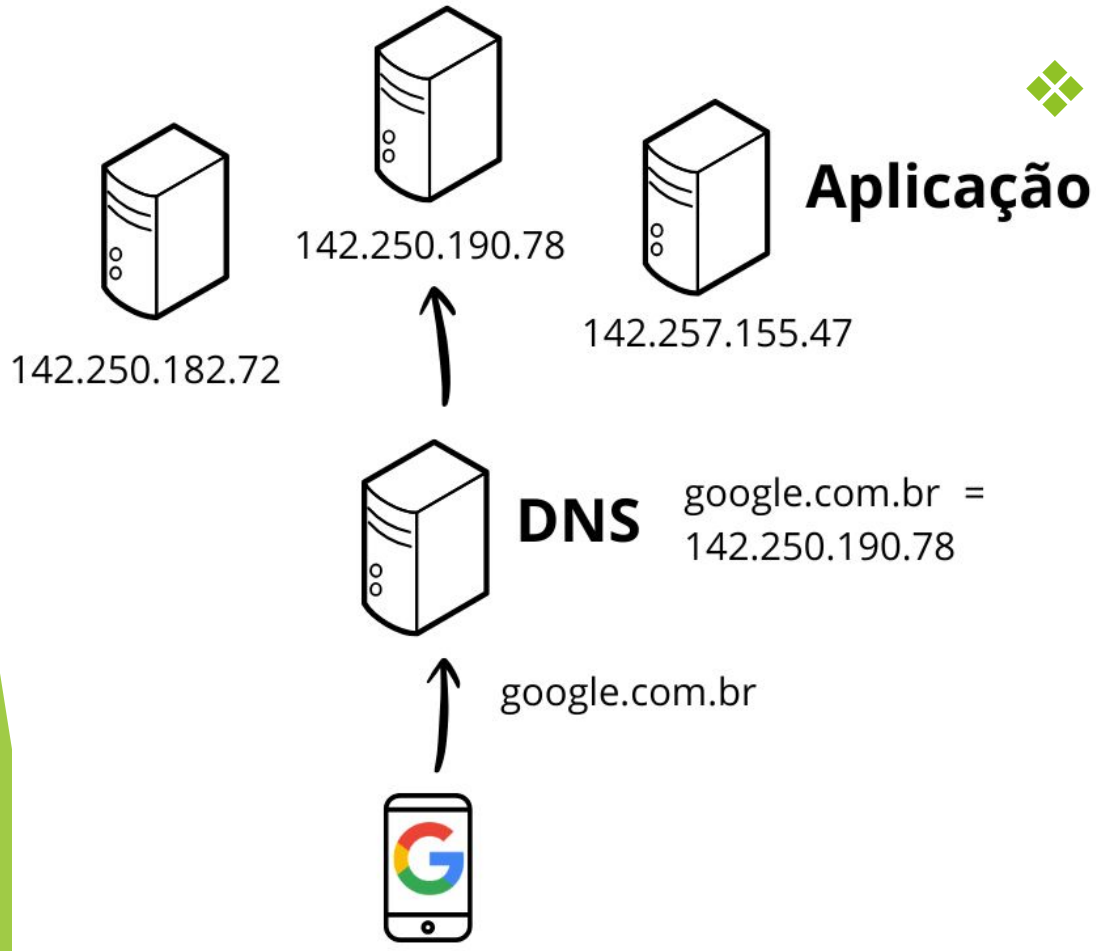
- Ele é incluído no sistema mundial de domínios, coordenada pela ICANN (Internet Corporation for Assigned Names and Numbers).
- Você ganha acesso para configurar os servidores DNS.
- Você aponta o domínio para um IP.

DNS (Domain Name System)

Fonte: Lucciani, 2025

- Significa Domain Name System ou **Sistema de Nomes de Domínio.**
- Ele funciona como uma agenda telefônica da Internet, **buscando o IP do domínio** acessado pelos usuários.

DNS (Domain Name System)



Fonte: autora, 2026

- Quando você digita: www.google.com.br, o DNS identifica o IP associado a esse domínio, por exemplo 142.250.190.78, e você é redirecionado ao servidor correto.

DNS (Domain Name System)

Mas porque precisamos do DNS?

- É muito mais fácil memorizar google.com.br, do que se ip, nesse exemplo, 142.250.190.78.
- Um mesmo servidor (com mesmo IP) pode armazenar vários sites, sendo redirecionado o acesso pro site correto pelo domínio indicado.
- Se o IP mudar, é somente atualizado o novo ip no servidor DNS, os usuários continuam acessando google.com.br.
- Redirecionamento entre múltiplos servidor, se uma aplicação está sendo processada em vários servidores, o dns distribui o tráfego.

Domínio x DNS

- O domínio é um **nome fácil de lembrar** associado a um site na internet.
- O sistema de DNS (Domain Name System) **converte esse nome em um endereço IP**, que é o identificador real do servidor na rede.
- Dessa forma, o domínio facilita o acesso dos usuários aos sites, evitando que seja necessário memorizar endereços IP.
- O DNS, portanto, é o sistema responsável por **traduzir nomes de domínio em endereços IP**.

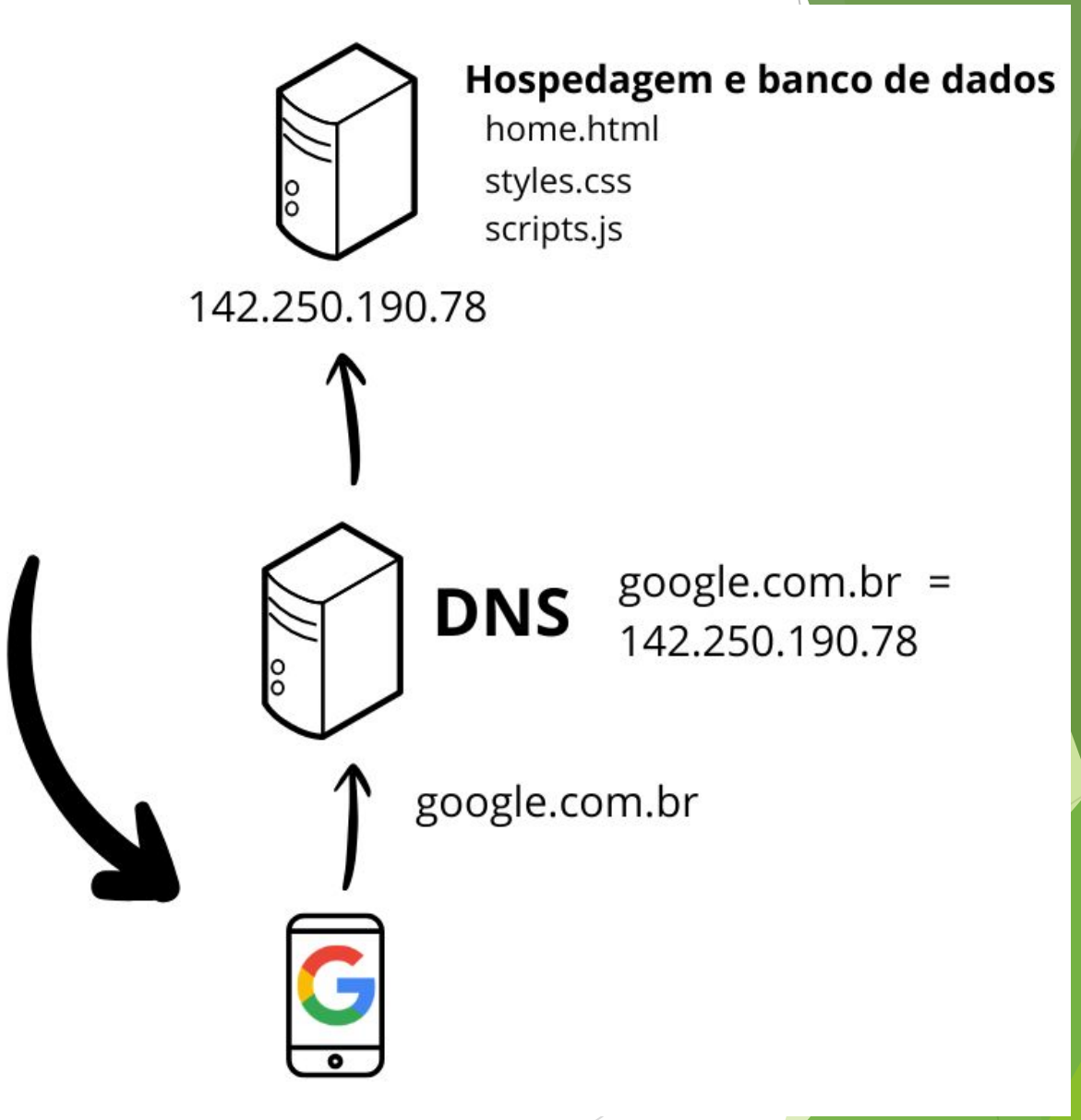
Hospedagem

- Quando criamos um site, ele precisa ficar armazenado em algum computador conectado à internet para que outras pessoas possam acessá-lo. Esse serviço é chamado de hospedagem de sites.

- A hospedagem consiste em **disponibilizar servidores conectados à internet que armazenam os arquivos** de um site e permitem que eles **sejam acessados por meio de um navegador**.

Hospedagem

- O usuário digita o endereço do site no navegador
- O DNS converte o domínio em um endereço IP
- O navegador se conecta ao servidor de hospedagem
- O servidor envia os arquivos do site ao navegador.



Fonte: autora, 2026

Hospedagens mais utilizadas para sites

Hospedagem compartilhada

- Vários sites utilizam o mesmo servidor, **compartilhando recursos** como: memória, processamento e armazenamento.
- É o tipo mais comum e mais barato com recursos limitados e desempenho variável.

VPS (Servidor Virtual Privado)

- Nesse modelo, um servidor físico é dividido em vários servidores virtuais.
- Cada site tem mais controle e recursos próprios. Tem mais desempenho e mais controle, e um custo mais elevado.

Hospedagens mais utilizadas para sites

Servidor dedicado

- Nesse caso, todo o servidor pertence a um único cliente, utilizando o desempenho máximo do servidor para si e tendo controle total. Exige administração técnica e tem custo elevado.

Hospedagem na nuvem

- Na hospedagem em nuvem, o site utiliza vários servidores conectados, o que aumenta a disponibilidade e a escalabilidade. Tem custo variável dependendo do uso, configurações complexas e dificuldade em prever custos.

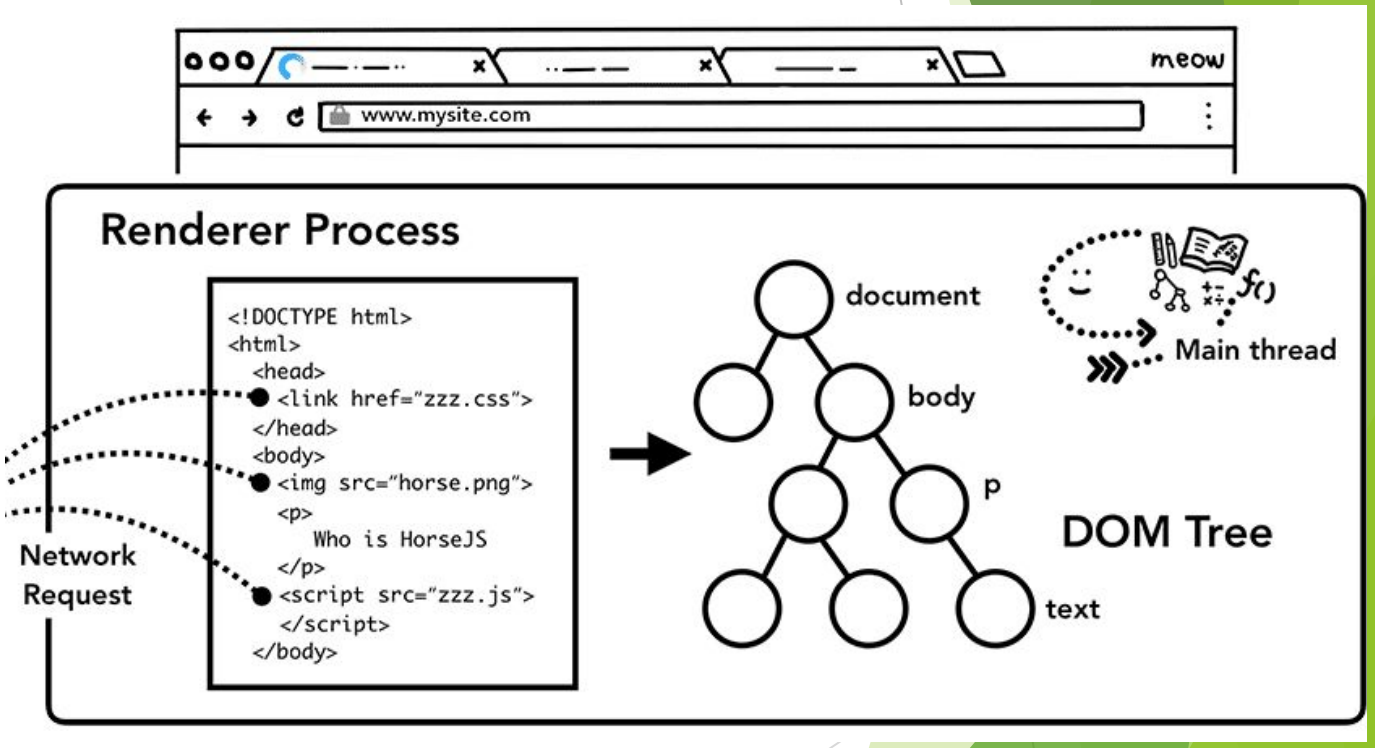
Navegadores (Browsers)

- Um navegador web é um software que permite **acessar** e **visualizar páginas da internet.**

Fonte: autora, 2026

Navegadores (Browsers)

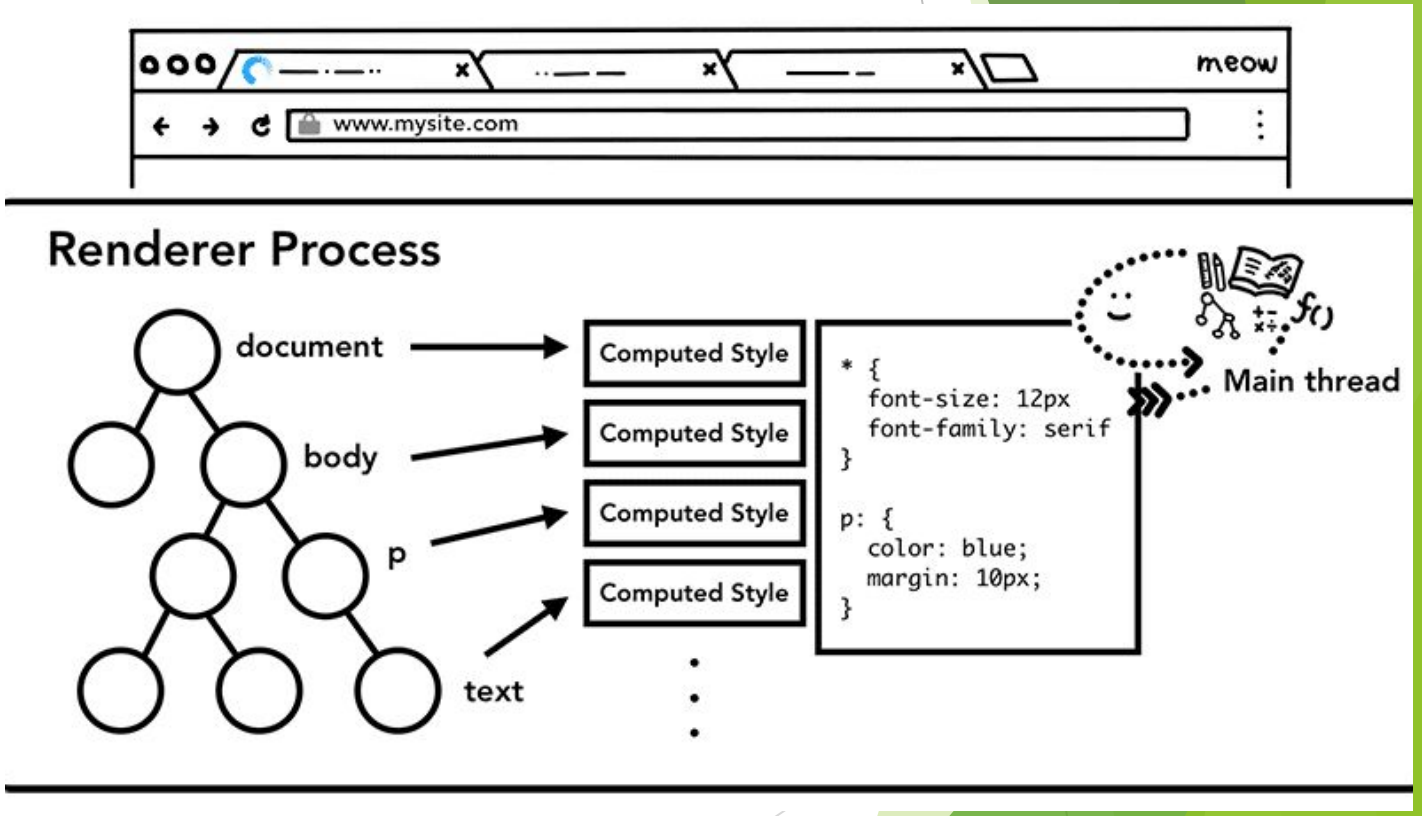
- Ele funciona como um **intermediário** entre o usuário e os servidores da web.
- **Interpretando os arquivos enviados pelo servidor** (HTML, CSS e JavaScript) e **exibindo o conteúdo de forma visual** na tela.



Fonte: Kosaka, 2018

Navegadores (Browsers)

- Desta forma, o navegador **transforma o código de um site em uma página** que podemos ler, clicar e interagir.



Fonte: Kosaka, 2018

URL (Uniform Resource Locator)

- A URL (Uniform Resource Locator) é o endereço completo de um recurso na internet.

Ela diz ao navegador:

- qual protocolo usar
- qual domínio acessar
- qual caminho dentro do servidor

URL (Uniform Resource Locator)

Fonte: MILETTO e BERTAGNOLLI, 2014

- Ele pode ser usado tanto na barra de endereços do navegador para acessar uma página ou usado como hiperlink, navegando para outra página.

Domínio x URL

- O domínio é como se fosse o "**nome fantasia**" de um site.

Em servidores dedicados, o domínio normalmente aponta para o IP exclusivo daquele servidor.

Em servidores compartilhados, vários domínios podem apontar para o mesmo IP. O servidor web identifica qual site deve ser exibido com base no domínio acessado pelo usuário.

- Já a URL é o **endereço completo** de um recurso no servidor.

Protocolo HTTP

Aula 3.3

HTTP (HyperText Transfer Protocol)

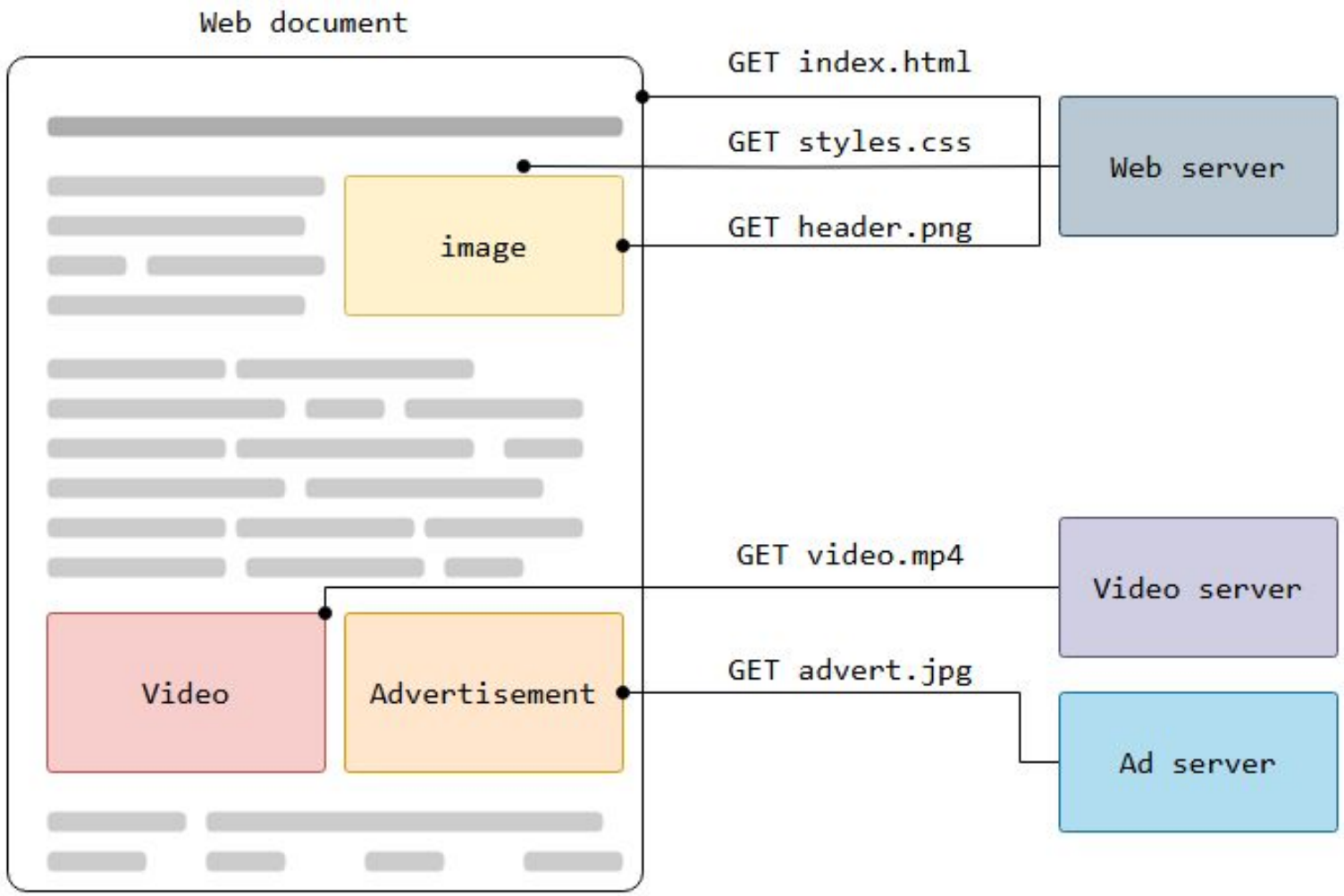
- O HTTP (HyperText Transfer Protocol) é o **protocolo** usado para **transferência de documentos hipermídia** (Hipertexto) na internet, e identifica o tipo de arquivo que será enviado. Surgiu em 1990.

- É a base de qualquer **troca de dados na Web** e um protocolo cliente-servidor, o que significa que as requisições são iniciadas pelo destinatário, geralmente um navegador da Web.

HTTP (HyperText Transfer Protocol)

- Para mostrar uma página Web, o navegador envia uma requisição para buscar o documento HTML da página.
- Ele então realiza uma análise sintática desse arquivo, realizando requisições para obter os recursos da página (geralmente imagens e vídeos).
- Depois o navegador interpreta esses recursos para mostrar ao usuário a página completa.

HTTP (HyperText Transfer Protocol)



Fonte: developer mozilla org

HTTP (HyperText Transfer Protocol)

- As mensagens enviadas pelo cliente, geralmente um navegador da Web, são chamadas de **solicitações** (**requests ou requisições**) e as mensagens enviadas pelo servidor são chamadas de **respostas** (**responses**).

- Frequentemente, o **agente-usuário** é um navegador da Web, mas pode ser qualquer coisa, como um robô que varre a Web para atualizar o índice de mecanismo de pesquisa.

Principais métodos HTTP

GET

- Usado para solicitar ou consultar um recurso no servidor.

Exemplo: carregar uma página ou buscar dados

GET /produtos

- Buscar dados
- Não altera o servidor

Principais métodos HTTP

POST

- Usado para enviar dados ao servidor, geralmente para criar um novo recurso.

Exemplo: cadastrar um usuário

POST /usuarios

- Criar dados
- Envia informações no corpo da requisição

Principais métodos HTTP

PUT

- Usado para atualizar completamente um recurso existente.

Exemplo: atualizar todas as informações de um usuário.

PUT /usuario/id/10

- Atualiza todos os dados do usuário de ID 10.
- Envia informações no corpo da requisição

Principais métodos HTTP

DELETE

- Usado para remover um recurso do servidor.

Exemplo: atualizar todas as informações de um usuário.

DELETE /usuario/id/10

- Remove o usuário de ID 10.

Outros métodos HTTP importantes

PATCH

- Usado para atualizar parcialmente um recurso.

Exemplo: alterar apenas o email de um usuário.

PATCH /usuarios/10

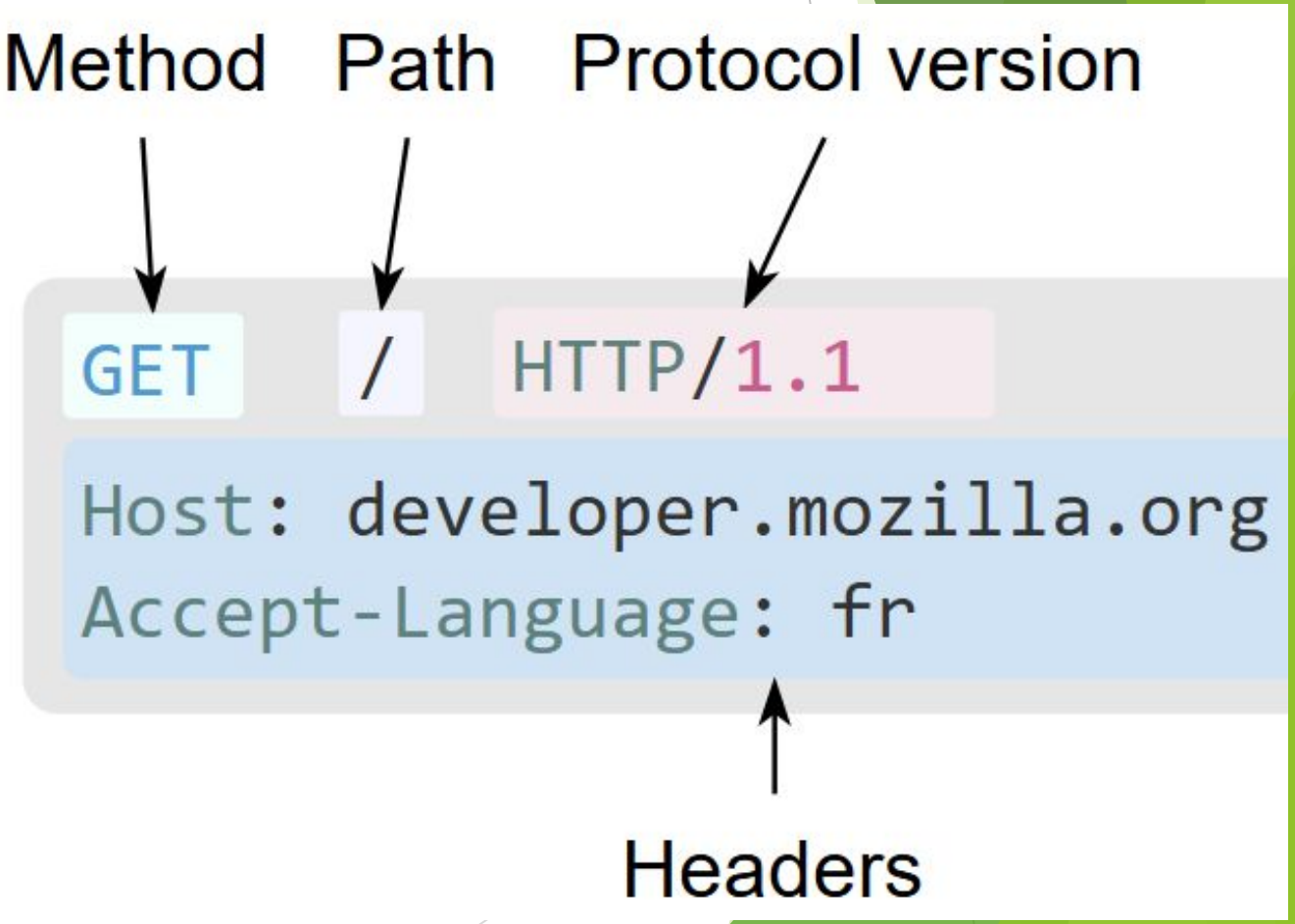
- Atualiza somente o dado do e-mail do usuário de ID 10.
- Envia informações no corpo da requisição

```
{
  "email": "novoemail@email.com"
}
```

Fonte: autora, 2026

HTTP: estrutura da requisição

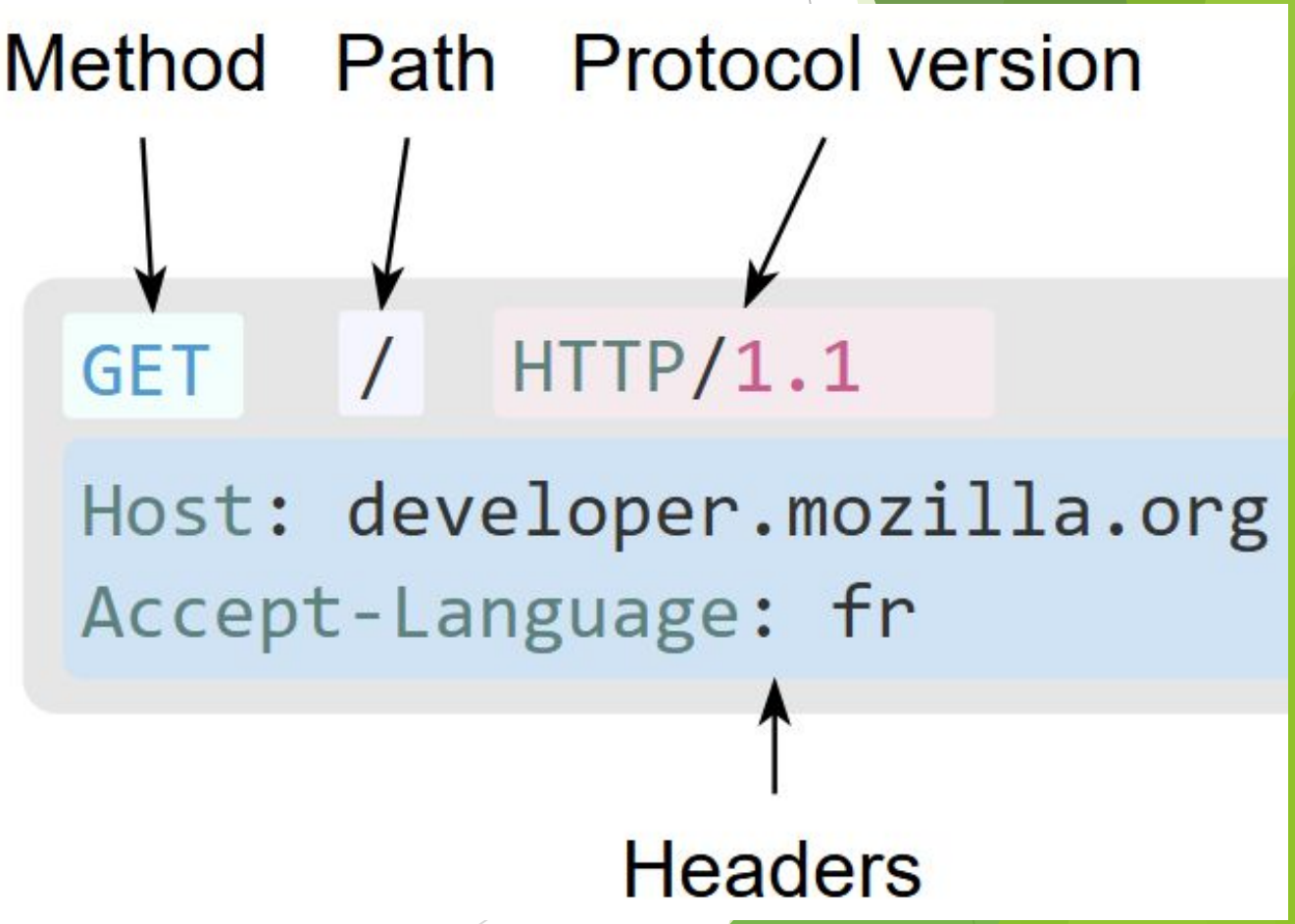
- **Método**: Geralmente é um verbo como GET, POST, DELETE, PUT ou substantivo como OPTIONS ou HEAD.



Fonte: developer mozilla org

HTTP: estrutura da requisição

- **Caminho**: A URL do recurso sem os elementos do contexto (protocolo, domínio e porta).

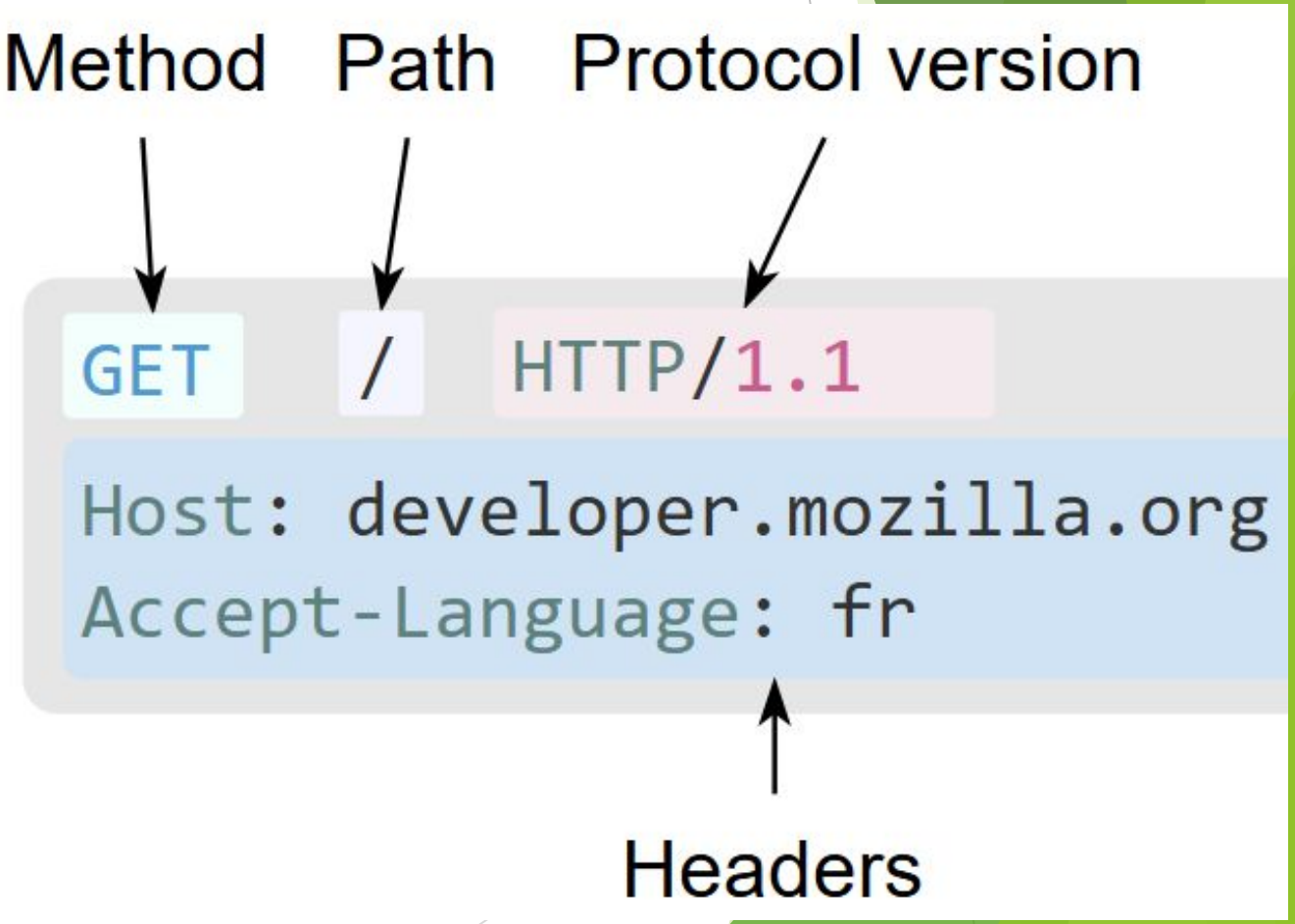


Fonte: developer mozilla org

HTTP: estrutura da requisição

- **Versão**: A versão do protocolo HTTP utilizada.

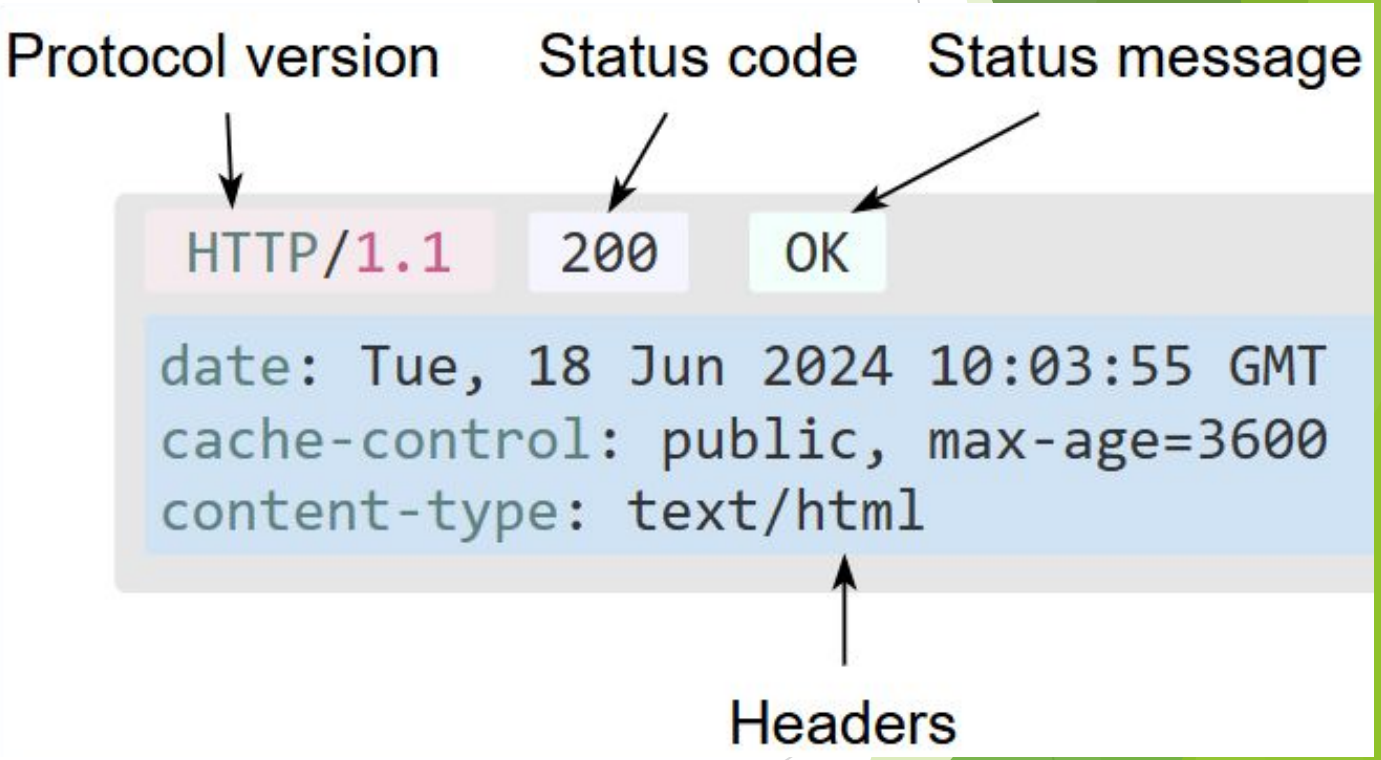
- Podemos ter informações opcionais, como os **cabeçalhos** que contém informações adicionais e o **conteúdo** que contém um corpo de dados, usado por métodos como POST.



Fonte: developer mozilla org

HTTP: estrutura da resposta

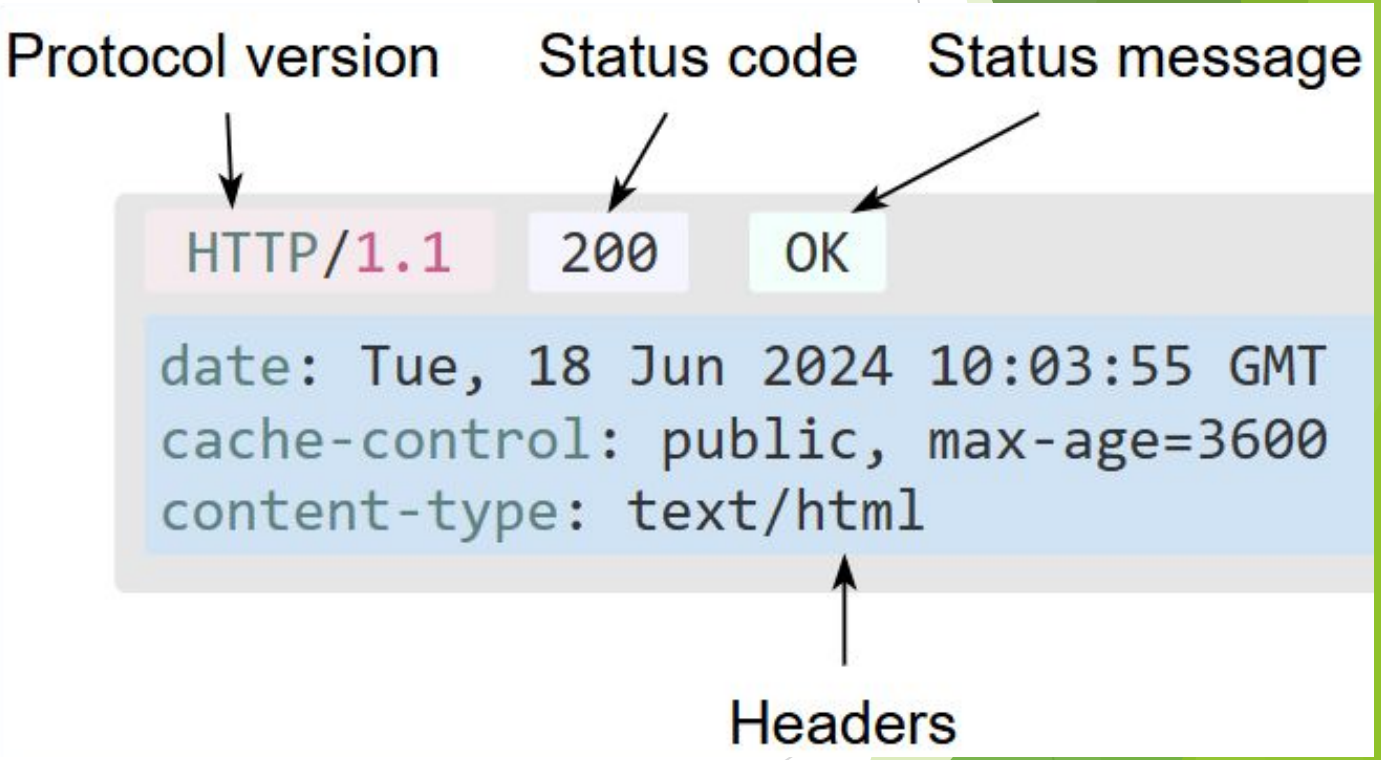
- **Versão**: A versão do protocolo HTTP utilizada.
- **Código de status**: códigos que indicam se a requisição foi bem sucedida ou não.
- **Mensagem de status**: pequena descrição do código de status.



Fonte: developer mozilla org

HTTP: estrutura da resposta

- Também podemos o campo de **cabeçalho** e o campo opcional de **corpo de dados**.



Fonte: developer mozilla org

Código de status das requisições HTML

Os códigos de status indicam se uma requisição HTTP foi executada com êxito ou não. Sendo agrupadas em 5 classes:

- Respostas Informativas (100 – 199)
- Respostas bem-sucedidas (200 – 299)
- Mensagens de redirecionamento (300 – 399)
- Respostas de erro do cliente (400 – 499)
- Respostas de erro do servidor (500 – 599)

Principais código de status de sucesso

200 OK

- A solicitação foi bem-sucedida.
- GET: O recurso foi obtido e transmitido no corpo da mensagem.
- HEAD: Os cabeçalhos de representação são incluídos na resposta sem nenhum corpo de mensagem.
- PUT ou POST: O recurso que descreve o resultado da ação é transmitido no corpo da mensagem.
- TRACE: O corpo da mensagem contém a mensagem de requisição recebida pelo servidor.

Principais código de status de sucesso

201 Created

- A requisição foi bem sucedida e um novo recurso foi criado como resultado.

- Esta é normalmente a resposta enviada após as solicitações POST ou algumas solicitações PUT.

Principais código de status de erro

400 Bad Request

- O servidor não pode ou não irá processar a solicitação devido um erro do cliente (por exemplo, sintaxe de solicitação malformada).

401 Unauthorized

- Embora o padrão HTTP especifique "unauthorized", semanticamente, essa resposta significa "unauthenticated".
- Ou seja, o cliente deve se autenticar para obter a resposta solicitada.

Principais código de status de erro

403 Forbidden

- O cliente <u>não tem direitos de acesso</u> ao conteúdo, não é autorizado, portanto o servidor está se recusando a fornecer o recurso solicitado.

- Ao contrário do 401 Unauthorized, a <u>identidade do cliente é conhecida</u> pelo servidor.

Principais código de status de erro

404 Not Found

- O servidor não pode encontrar o recurso solicitado.
- No navegador, isso significa que o URL não é reconhecido.
- Em uma API, isso também pode significar que o endpoint é válido, mas o próprio recurso não existe.
- Os servidores também podem enviar esta resposta em vez de 403 Forbidden para ocultar a existência de um recurso de um cliente não autorizado.

Principais código de status de erro

405 Method Not Allowed

- O método de solicitação é conhecido pelo servidor, mas não é suportado pelo recurso de destino.

Ocorre quando:

- O servidor reconhece o método HTTP utilizado.
- Porém esse método não é permitido para aquele recurso específico.
- Exemplo: o endpoint é um get, mas tento passar como delete

Principais código de status de erro

429 Too Many Requests

- Indica que o cliente enviou requisições demais ao servidor em um período muito curto de tempo.
- Para evitar sobrecarga ou abuso do sistema, muitos servidores implementam um mecanismo chamado limitação de frequência (rate limiting).
- Esse mecanismo define quantas requisições um usuário ou aplicação pode fazer em determinado intervalo de tempo.

Principais código de status de erro

500 Internal Server Error

- O erro HTTP 500 (Internal Server Error) indica que ocorreu um problema inesperado no servidor, e ele não conseguiu processar a requisição feita pelo cliente.

- O servidor tentou executar a operação solicitada, mas encontrou um erro interno que não conseguiu tratar adequadamente.

Principais código de status de erro

501 Not Implemented

- Indica que o servidor não reconhece ou não suporta o método HTTP utilizado na requisição. Por esse motivo, ele não consegue processar a solicitação.

- Se um cliente usar um método que não foi implementado ou não é suportado pelo servidor, ele pode responder com 501 Not Implemented.

Métodos que todo servidor deve suportar

De acordo com a especificação do protocolo HTTP, todo servidor deve ser capaz de lidar pelo menos com os métodos:

- GET
- HEAD

Por isso, um servidor não deveria retornar 501 para esses dois métodos, pois eles são considerados básicos para o funcionamento da web.

Principais código de status de erro

503 Service Unavailable

- Indica que o servidor está temporariamente indisponível para processar a requisição.
- Isso acontece quando o servidor está funcionando, mas não consegue atender à solicitação naquele momento.
- Normalmente esse problema é temporário, e o serviço volta a funcionar depois de algum tempo.

Principais código de status de erro

O erro 503 pode ocorrer por vários motivos, como:

- Manutenção do servidor
- Sobrecarga de acessos (muitos usuários ao mesmo tempo)
- Recursos insuficientes (memória, CPU ou conexões)
- Reinicialização ou atualização do sistema

Dúvidas?

Exemplo prático: API da Nasa

Software utilizado para as requisições: Insomnia

Link para as APIs Públicas da NASA: NASA OPEN API

APIs:

- APOD: foto astronômica do dia: link
- Asteroids - NeoWs > Neo - Feed: Listagem de asteróides de acordo com sua proximidade da terra: link
- InSight: Mars Weather Service API: link

Atividade 3

- Responder questionário referente aos assuntos da aula de hoje.
- **Data de entrega**: até o final da aula de HOJE
- **Forma de entrega**: manuscrito físico.
- **Execução**: duplas (colocar nome de todos)
- **Fonte de consulta**: material da aula, seu colega e internet.

Referência da aula

- LUCCIANI, Geraldo. Quanto tempo levam as alterações de DNS no Registro.br? Veja. TASK, 2025. Disponível em: https://www.task.com.br/blog/quanto-tempo-demora-alteracoes-registro-br/. Acesso em: 5 mar. 2026.

- Mariko Kosaka. Inside look at modern web browser (part 3). Google, 2018. Disponível em: https://developer.chrome.com/blog/inside-browser-part3?hl=pt-br. Acesso em: 5 mar. 2026.

- MOZILLA. Códigos de status de respostas HTTP. MDN Web Docs. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status. Acesso em: 5 mar. 2026.

univille

univille.br